

JournalMind: An AI-Powered Reflective Journaling Assistant

(An LLM Assistant for Enhanced Personal Journaling)

Abhik Sarkar

Student ID: 24214927

Email: abhik.sarkar@ucdconnect.ie

Yash Singh Pathania

Student ID: 24204265

Email: yash.pathania@ucdconnect.ie

COMP47980: Generative AI And Language Models

Project Report, 2025

Table of Contents

List of Figures	3
1 Introduction	4
2 Assistant Design: A Schematic View	6
3 Added Value: More than Mere ChatGPT or basic LLM	9
3.1 Tool Usage and Advanced Retrieval Pipeline	10
4 Outside Knowledge: Curated Data Sources	14
5 Worked Examples: Your Assistant in Action	15
6 Summary and Conclusions	19
7 Acknowledgements	20
8 References	20

List of Figures

1	Evaluation of various AI agents.	5
2	High-Level Architecture of JournalMind, showing Frontend, Backend, Databases, and AI Integration Layer.	8
3	Example Interaction: User asking about yesterday's mood.	16
4	Example Interaction: User asking about interactions with 'Dr Sharma'.	17
5	Example: A journal entry showing the AI-generated title and image.	18

1 Introduction

This report presents the design, implementation, and evaluation of **JournalMind**, an innovative personal journaling assistant leveraging Large Language Models (LLMs). While traditional journaling practices typically involve passive documentation of events and emotions, extracting meaningful insights from accumulated entries often proves challenging. JournalMind transforms this experience through active engagement with user content, delivering sophisticated analysis, comprehensive summaries, and contextually aware interactions.

Product Overview

JournalMind is a sophisticated web application that enables users to create daily journal entries, organize them with tags, and engage with an intelligent AI assistant. The platform offers several distinctive features:

- Robust user authentication implemented via Google OAuth2 protocols.
- Intuitive management of chronological journal entries (limited to one per day).
- Advanced AI-driven sentiment analysis with automatic emotion tagging.
- Conversational AI assistant capable of interrogating past entries, analyzing emotional patterns, generating comprehensive summaries, and providing insights about specific individuals mentioned.
- Automated generation of creative, F.R.I.E.N.D.S.-inspired episode titles for each entry.
- AI-powered visualization through custom cover images that reflect entry content.

The assistant has been meticulously designed to provide insightful, reflective, and helpful interactions, specifically tailored to each user's documented personal history.

Value Proposition

The fundamental motivation behind JournalMind is to enhance self-reflection and deepen personal understanding. Standard LLMs such as ChatGPT exhibit significant limitations—they lack persistent memory of individual user context across sessions and cannot effectively perform structured analysis over an extensive corpus of personal, private text. JournalMind addresses these limitations by:

- Establishing a secure, persistent, and private repository for journal entries.
- Harnessing AI capabilities to identify patterns, emotional trends, and significant events that might otherwise remain undetected.
- Facilitating novel engagement with past experiences through comprehensive summaries and interactive Q&A.
- Incorporating creative elements (custom titles and images) to enhance user engagement.
- Delivering substantial value beyond generic chatbot capabilities within this specialized personal domain.

The ultimate objective is to create an assistant that comprehensively understands the user's documented experiences, offering uniquely personalized insights unavailable through alternative platforms.

Technical Architecture

JournalMind employs a modern web architecture with a React frontend and FastAPI backend. User data, including journal entries and associated metadata, is stored in PostgreSQL with proper encryption and access controls. For semantic search and AI-powered insights, entries are embedded and indexed in Qdrant vector database. The platform integrates the OpenAI Assistants API, specifically utilizing the gpt-4o-mini model (selected through rigorous evaluation potentially employing ChainForge), to power its advanced AI features. The assistant leverages several sophisticated techniques:

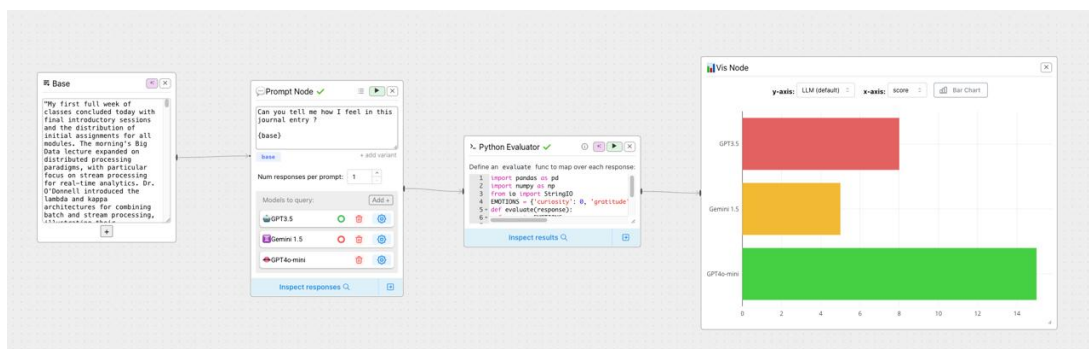


Figure 1: Evaluation of various AI agents.

- **Custom Retrieval-Augmented Generation (RAG):** Journal entries are systematically indexed in a vector database, enabling the assistant to retrieve contextually relevant historical content to inform its responses through a sophisticated multi-layered retrieval pipeline.

- **Function Calling:** The Assistants API's function calling capabilities are utilized to execute specific, structured tasks such as generating monthly summaries or analyzing patterns in interactions with individuals mentioned throughout the journal.
- **Strategic Prompt Engineering:** Meticulously crafted system instructions and prompts guide the LLM's personality, analytical approach, and creative outputs (including title generation).

This integrated approach enables JournalMind to function as a deeply personalized, contextually aware journaling companion that evolves with the user's documented life experiences.

2 Assistant Design: A Schematic View

JournalMind utilizes a client-server architecture, separating the user interface (React) from the backend logic and data management (FastAPI). The core AI capabilities are enabled by the OpenAI Assistants API, integrated through a well-defined service layer.

High-Level Architecture:

- **Frontend:** A React Single-Page Application (SPA) built with Vite, using JavaScript and styled with TailwindCSS. It handles user interactions, displays journal entries and AI responses, and communicates with the backend via RESTful API calls (`frontend/src/api.js`).
- **Backend:** A FastAPI server written in Python (`backend/app/main.py`). It manages API endpoints (authentication, journal CRUD, assistant interactions, insights), business logic, database interactions, and AI service coordination.
- **Data Storage:**
 - PostgreSQL: Stores structured data like user information, journal entries (text, date, tags), and periodic summaries, managed via Tortoise ORM (`backend/app/models.py`).
 - Vector Database: Stores embeddings of journal entries for efficient semantic search, enabling the custom RAG functionality (`backend/app/utils/vector_db_utils.py`).
 - Redis: Used for caching, particularly for managing Google OAuth access tokens.
- **AI Integration Layer:**
 - AIManager: (`backend/app/utils/ai/manager.py`): A singleton class acting as a facade for AI operations, abstracting the specific provider.

- **OpenAIProvider:** (`backend/app/utils/ai/providers.py`): Implements the interaction logic specifically for the OpenAI Assistants API (`gpt-4o-mini`).
 - **AssistantService:** (`backend/app/services/assistant_service.py`): Manages the lifecycle of assistants, threads, messages, and runs, including context injection from the vector database and handling function calls.
 - **JournalService:** (`backend/app/services/journal_service.py`): Contains business logic integrating AI features like title/image generation and sentiment analysis into the journaling workflow.
 - **Prompts Config:** (`backend/app/config/prompts.py`): Centralizes system instructions and task-specific prompts.
- **Containerization:** Docker is used for packaging and deployment consistency.

Leveraged Assistants API Features:

- **Assistants, Threads, Messages, Runs:** The core objects of the API are used to manage the conversational state and interaction flow with the `gpt-4o-mini` model. Each user typically has a persistent thread associated with their account.
- **Function Calling:** This is heavily utilized to allow the assistant to perform actions beyond text generation, such as retrieving specific data summaries (`get_monthly_summary`, `get_important_events`) or analyzing specific entities (`get_friend_personality`). These functions are defined in the assistant's configuration and implemented in the backend.
- **Model Choice:** `gpt-4o-mini` is used as the underlying LLM, chosen for its balance of capability and cost-effectiveness.

Features Not Used (and Why):

- **Built-in File Search (RAG):** JournalMind implements a custom RAG mechanism using its own vector database. This provides greater control over the indexing, retrieval process, and integration with the application's primary PostgreSQL database, which is essential for grounding the AI in the user's specific journal entries.
- **Code Interpreter:** The primary focus is analysis, and generation based on journal text. Complex calculations, data visualization, or code execution are not core requirements, making the Code Interpreter tool unnecessary for the current feature set.

This design allows for a specialized assistant deeply integrated with the user's personal data, going beyond the capabilities of a general-purpose chatbot.

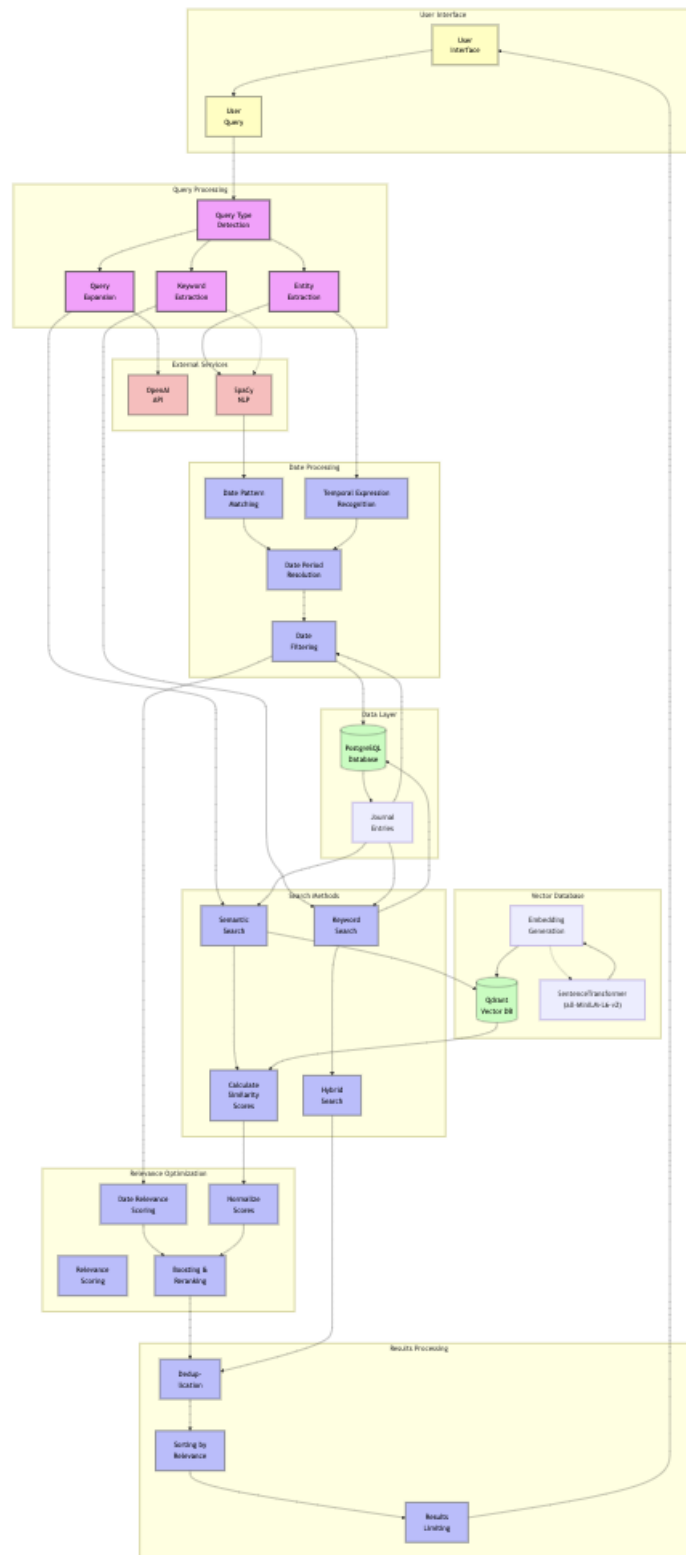


Figure 2: High-Level Architecture of JournalMind, showing Frontend, Backend, Databases, and AI Integration Layer.

3 Added Value: More than Mere ChatGPT or basic LLM

Standard implementations of Large Language Models, whether through interfaces like ChatGPT or basic API calls, fail to address JournalMind's specialized requirements. General-purpose conversational AI lacks the domain-specific context management, persistent memory architecture, and purpose-built analytical tools necessary to function as a truly personalized journaling companion. JournalMind demands a tailored implementation that maintains longitudinal awareness across entries, applies specialized psychological frameworks, and delivers insights with contextual relevance that generic LLM deployments cannot achieve.

Limitations of Vanilla LLMs/ChatGPT for Journaling:

- **Lack of Persistent Context:** ChatGPT sessions are generally ephemeral or have limited context windows. It cannot inherently "remember" the user's entire journal history across multiple interactions or days without manual re-feeding of information.
- **Inability to Query Structured Data:** It cannot easily perform targeted searches or analyses across a large corpus of entries (e.g., "Find all entries where I mentioned feeling anxious about work" or "Summarize my interactions with Jane last month").
- **Generic Responses:** Without specific grounding in the user's data and tailored instructions, responses tend to be generic rather than deeply personalized.
- **Lack of Integrated Tools:** Generating entry-specific images or performing specific data retrieval tasks requires external tools or complex prompting sequences not native to the basic chat interface.

How JournalMind Transcends These Limits: JournalMind's design choices directly address these limitations to provide significant added value:

- **Advanced Multi-layered Retrieval Pipeline:** This sophisticated system forms the foundation of JournalMind's personalization capabilities, going far beyond basic RAG implementations. By combining multiple search methodologies, query processing techniques, and contextual understanding, the system can intelligently retrieve the most relevant journal entries regardless of how users phrase their queries.
- **Strategic Use of Function Calling:** Function calling enables the assistant to perform specific, pre-defined actions that require querying the database or applying custom logic. Functions like `get_monthly_summary`, `get_important_events`, and `get_friend_personality` allow users to ask complex questions that translate into structured operations on their journal data, yielding specific, synthesized answers rather than just generative text.

- **Tailored Prompt Engineering:** System instructions (`backend/app/config/prompts.py`) define the assistant's persona (insightful, reflective) and its core tasks (analyze entries, provide insights, generate creative content). Specific prompts for features like F.R.I.E.N.D.S. title generation ensure consistent and desired outputs.
- **Integrated Workflow Automation:** Features like automatic sentiment/emotion tagging upon entry creation, periodic summary generation, and the two-stage image generation process automate tasks that would be cumbersome or impossible to achieve consistently through manual interaction with a generic LLM.

Together, these elements create an assistant that is not just a conversationalist but a specialized tool for self-reflection, deeply integrated with the user's unique data and capable of performing tasks specifically designed for the journaling domain.

3.1 Tool Usage and Advanced Retrieval Pipeline

JournalMind strategically utilizes the tools offered by the OpenAI Assistants API, alongside custom implementations, to achieve its functionality. The primary API tool leveraged is Function Calling, but the most significant technical contribution is the Advanced Retrieval Pipeline.

Function Calling: This feature is critical for enabling the assistant to perform structured actions based on the user's journal content. Instead of just generating text, the assistant can request the execution of specific backend functions.

- **Why it's used:** To bridge the gap between the LLM's understanding and the need for specific data retrieval or processing from the application's database and logic. It allows for complex, data-driven queries to be answered accurately.
- **Implemented Functions:**
 - `get_monthly_summary`: Retrieves and summarizes journal entries from a specific recent period (e.g., the last month).
 - `get_important_events`: Analyzes recent entries to identify and list significant events reported by the user.
 - `get_friend_personality`: Searches across entries for mentions of a specific person and synthesizes information about the user's interactions with or feelings towards them.
- **How it works:** These functions are defined in the Assistant's configuration. When the LLM determines one of these functions is needed to answer a user query, it outputs

a request to call the function. The Assistant Service intercepts this, executes the corresponding Python code (which typically queries the database or vector store), and sends the result back to the LLM to formulate the final response.

Advanced Retrieval Pipeline: JournalMind’s sophisticated retrieval system is a multi-layered architecture designed to intelligently find relevant journal entries. This custom-built system surpasses the capabilities of the Assistants API’s built-in `file_search` tool by integrating advanced natural language processing (NLP), vector-based semantic search, and precise date handling to deliver highly relevant results tailored to user queries.

- **Query Processing Layer:** This layer preprocesses and interprets user queries to enable precise retrieval.
 - **Query Type Detection:** Analyzes the query to determine its intent—whether it pertains to dates, emotions, topics, or people. This is achieved using NLP techniques, including rule-based keyword matching and part-of-speech tagging implemented with SpaCy. For example, a query like “How did I feel last month?” triggers detection of an emotion-related intent due to “feel” and a date-related intent due to “last month.”
 - **Entity Extraction:** Employs SpaCy’s transformer-based named entity recognition (NER) model (`en_core_web_trf`) to identify key entities such as dates, people, locations, and organizations. For instance, in “Meeting with Alice on 01/15/2023,” “Alice” is tagged as PERSON and “01/15/2023” as DATE.
 - **Query Expansion:** Utilizes OpenAI’s `gpt-4o-mini` model to enrich the query with semantically related terms, improving matching accuracy. The model is prompted with instructions like “Generate synonyms and related concepts for: [query],” transforming “happy” into a set including “joy,” “contentment,” and “satisfaction.”
 - **Keyword Extraction:** Identifies significant terms by filtering out stopwords using NLTK’s stopwords list, retaining only meaningful words (e.g., nouns, verbs) for downstream processing.
- **Vector Embeddings & Semantic Search Layer:** This layer enables meaning-based retrieval through vector representations.
 - **Embedding Generation:** Converts queries and journal entries into 384-dimensional vectors using the `all-MiniLM-L6-v2` model from SentenceTransformers. This distilled transformer model is trained on large corpora to capture semantic similarity efficiently.

- **Vector Database:** Stores embeddings in Qdrant, a vector database optimized for fast similarity searches using the HNSW (Hierarchical Navigable Small World) algorithm, enabling approximate nearest neighbor retrieval with low latency.
- **Semantic Search:** Identifies entries with similar meanings by comparing query and entry vectors, allowing retrieval of conceptually related content (e.g., "stress" entries for an "overwhelmed" query).
- **Cosine Similarity:** Quantifies vector similarity using the formula:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

where $\mathbf{A}$ and $\mathbf{B}$ are the query and entry vectors, respectively, yielding scores from -1 (opposite) to 1 (identical).

- **Date Processing Layer:** Handles temporal aspects of queries for precise filtering.
 - **Date Pattern Matching:** Employs regular expressions (regex) to detect explicit date formats, such as `\d{1,2}/\d{1,2}/\d{4}` for MM/DD/YYYY (e.g., "01/15/2023") or `\d{4}-\d{2}-\d{2}` for YYYY-MM-DD.
 - **Temporal Expression Recognition:** Parses natural language phrases like "yesterday" or "next month" using the `dateutil` library, which converts them into actionable date references relative to the current date.
 - **Date Period Resolution:** Transforms relative expressions into absolute date ranges. For example, "last week" resolves to the specific start and end dates of the prior week based on the system's current timestamp.
 - **Date Filtering:** Filters journal entries to include only those within the resolved date range, ensuring temporal relevance in the results.
- **Multiple Search Methods Layer:** Combines diverse retrieval strategies for comprehensive results.
 - **Semantic Search:** Retrieves entries based on vector similarity, as described above, focusing on meaning over exact matches.
 - **Keyword Search:** Implements exact word matching using the BM25 algorithm, a probabilistic ranking function that scores entries based on term frequency (TF) and inverse document frequency (IDF), prioritizing documents with query terms.
 - **Hybrid Search:** Integrates semantic and keyword search results via a weighted combination, such as:

$$\text{hybrid_score} = \alpha \cdot \text{semantic_score} + (1 - \alpha) \cdot \text{keyword_score}$$

where α (e.g., 0.7) balances the contribution of each method, tuned for optimal performance.

- **Relevance Optimization Layer:** Refines result rankings for maximum relevance.
 - **Relevance Scoring:** Computes a composite score for each entry, incorporating the hybrid search score and additional factors like date relevance. A possible formula is:
$$\text{final_score} = \text{hybrid_score} + \beta \cdot \text{date_relevance}$$
where β weights the temporal factor.
 - **Score Normalization:** Normalizes scores to a [0,1] range using min-max scaling, ensuring comparability across methods.
 - **Date Relevance:** Assigns a binary or weighted score (e.g., 1 for matches, 0 for non-matches) to boost entries within the query's date range.
 - **Boosting & Reranking:** Applies additional adjustments, such as boosting recent entries or user-preferred topics, and reorders results based on the final score.
- **Results Processing Layer:** Polishes the output for delivery to the user.
 - **Deduplication:** Eliminates duplicate entries retrieved by multiple methods by tracking unique entry identifiers (e.g., IDs or content hashes), ensuring each entry appears only once.
 - **Sorting:** Orders entries in descending order of their final relevance scores, prioritizing the most relevant results.
 - **Result Limiting:** Restricts output to the top k entries (e.g., $k = 10$), delivering a concise yet pertinent set of results.

This multi-layered approach ensures that regardless of how users phrase their queries, the system identifies the most relevant journal entries by simultaneously understanding meaning, context, dates, and keywords. By leveraging state-of-the-art NLP, vector search, and hybrid techniques, JournalMind provides a foundation for truly personalized and intelligent AI interactions.

Code Interpreter: This tool is not used in JournalMind.

- **Why it's not used:** The core functionality revolves around natural language processing, text analysis, retrieval from databases, and creative generation based on text. There is no current requirement for the assistant to perform complex mathematical calculations, run simulations, analyze data files provided by the user, or generate visualizations, which are the primary use cases for the Code Interpreter. The necessary logic is handled by the backend Python code and database queries.

This selective and adapted use of tools allows JournalMind to leverage the strengths of the Assistants API while maintaining control and flexibility through custom implementations where necessary.

4 Outside Knowledge: Curated Data Sources

The primary source of "outside knowledge" that distinguishes JournalMind from a generic LLM is the user's own corpus of journal entries. This data is curated and managed entirely within the application ecosystem.

- **Where does it come from?:** Directly from user input through the JournalMind application interface. Each entry is created and saved by the user.
- **How much is there?:** The volume of data is variable and depends entirely on the user's activity. It can range from a few entries to potentially thousands over time.
- **How is it cleaned/preprocessed?:**
 - Entries are stored as text in the PostgreSQL database.
 - Upon creation or update, each entry undergoes:
 - * **Sentiment Analysis & Emotion Tagging:** An AI call (via the AIManager) processes the text to determine overall sentiment (positive, negative, neutral) and extract relevant emotion tags (e.g., happy, anxious, grateful). This metadata is stored alongside the entry.
 - * **Embedding:** The text content is converted into a numerical vector representation (embedding) using SentenceTransformer (all-MiniLM-L6-v2), producing a 384-dimensional vector that captures the semantic meaning of the entry.
 - * **Indexing:** The embedding and entry ID are stored in the Qdrant vector database for efficient semantic search.
- **What kind of special expertise does it provide?:** This data source provides unparalleled expertise on the user's personal life, history, thoughts, feelings, relationships, and experiences as documented by them. It allows the assistant to have deep, longitudinal context specific to that individual user, enabling truly personalized insights and interactions.

Example Data Snippet Structure: A single data point conceptually consists of:

- Entry ID: Unique identifier

- User ID: Link to the user
- Date: Date of the entry
- Content: "Had a long day at work, project deadline is stressful. Met Jane for coffee afterwards which was nice. Felt anxious but also grateful for the chat." (User's text)
- Tags: [User-defined tags, e.g., 'work', 'friends']
- AI-Generated Sentiment: Negative
- AI-Generated Emotions: ['stressful', 'anxious', 'nice', 'grateful']
- Embedding Vector: [Numerical vector representation...] (Stored in Qdrant)

This curated, user-specific knowledge base, accessed via the advanced retrieval pipeline, is what allows JournalMind to provide its core value proposition.

5 Worked Examples: Your Assistant in Action

To illustrate JournalMind's capabilities and the value added by its design, here are a few examples of typical interactions, highlighting the underlying mechanisms.

Example 1: Simple Mood Check

- **User Input:** "How was I feeling on 5th August?"
- **JournalMind Process:**
 - The advanced retrieval pipeline's date processing layer identifies "5th August" and converts it to an actual date.
 - The query type detection recognizes this as a temporal query about emotions.
 - It retrieves the journal entry for that date from PostgreSQL, including the pre-calculated sentiment and emotion tags.
 - The vector search component may find semantically similar entries discussing the same feelings.
 - The entry text and metadata are passed to the `gpt-4o-mini` assistant via the `AssistantService`.
 - The LLM synthesizes the information into a natural language response.

- **Assistant Output (Example):** "On August 5th, you seemed to have a bit of a stressful day, mentioning anxiety about a project deadline. However, you also felt grateful after having coffee with Jane, which you described as nice."
- **Value Add vs. ChatGPT:** ChatGPT would require the user to manually find and paste the entry from August 5th. JournalMind accesses the specific, relevant data automatically through its sophisticated date processing layer and leverages the pre-calculated sentiment analysis.

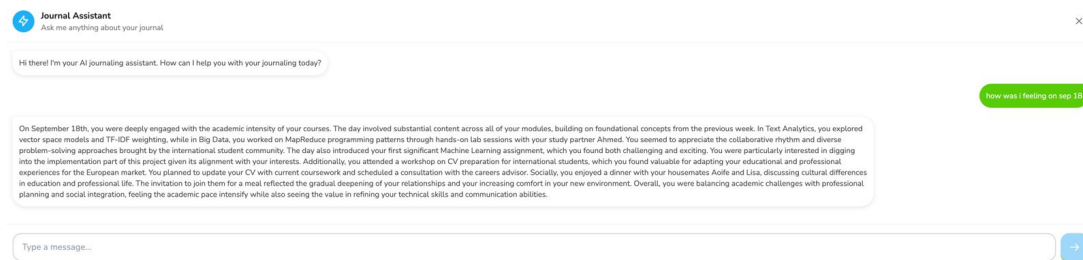


Figure 3: Example Interaction: User asking about yesterday's mood.

Example 2: Analyzing Interactions with a Person

- **User Input:** "What have my recent interactions with Dr Sharma been like?"
- **JournalMind Process:**
 1. The query is processed through the advanced retrieval pipeline:
 - Entity extraction identifies "Dr Sharma" as a person entity
 - Query expansion enriches the query with terms like "conversations," "meetings," "feelings about Dr Sharma"
 - The temporal term "recent" is identified and interpreted as a relative date range
 2. The Assistants API takes the processed query and recognizes the need for specific information about "Dr Sharma."
 3. The LLM determines the `get_friend_personality` function is appropriate.
 4. The Assistants API returns a request to execute `get_friend_personality` with the argument `name="Dr Sharma"`.
 5. The `AssistantService` executes the function, which triggers the advanced retrieval pipeline:
 - Multiple search methods (semantic, keyword, hybrid) find entries mentioning "Dr Sharma"
 - Results are scored, reranked, and deduplicated

- The most relevant entries are extracted from PostgreSQL
 - 6. The function processes the retrieved entries to analyze sentiment, extract key interactions, and identify patterns.
 - 7. The summary/analysis result is sent back to the Assistant.
 - 8. The LLM uses this result to formulate the final response.
- **Assistant Output (Example):** "Based on your entries from the past few weeks, your interactions with Dr Sharma seem mostly positive, particularly around the shared project success. However, you did mention feeling a bit frustrated during the planning meeting on the 15th."
 - **Value Add vs. ChatGPT:** This requires structured search across multiple entries using entity extraction, semantic search, and temporal understanding. ChatGPT could only do this if the user manually found and pasted all relevant snippets. JournalMind automates this complex retrieval and analysis using its advanced pipeline and Function Calling.

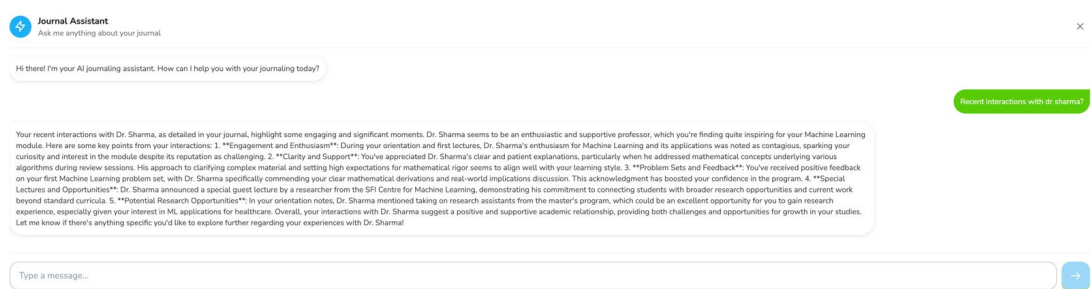


Figure 4: Example Interaction: User asking about interactions with 'Dr Sharma'.

Example 3: Entry Creation with Title and Image

- **User Input:** User saves a new journal entry about a grade he received and connections he created.
- **JournalMind Process:**
 1. The entry is saved to PostgreSQL and processed:
 - Sentiment/emotion analysis is performed (e.g., detecting "proud," "connected," "happy") and saved.
 - The entry is embedded using SentenceTransformer (all-MiniLM-L6-v2).
 - The resulting 384-dimensional vector is indexed in Qdrant.
 2. The JournalService triggers the title generation feature. A specific prompt instructs the LLM (via AIManager) to create a F.R.I.E.N.D.S.-style title based on the entry content.

3. The `JournalService` triggers the image generation feature. This involves a two-stage AI process:
 - Generate a detailed descriptive prompt for an image based on the journal entry content.
 - Use this generated prompt to call an image generation model (e.g., DALL-E 3 via API) to create the cover image.
 4. The generated title and image are associated with the journal entry.
- **Assistant Output (Displayed on Entry):** * Title: "The One With The A Grade and Global Connections"

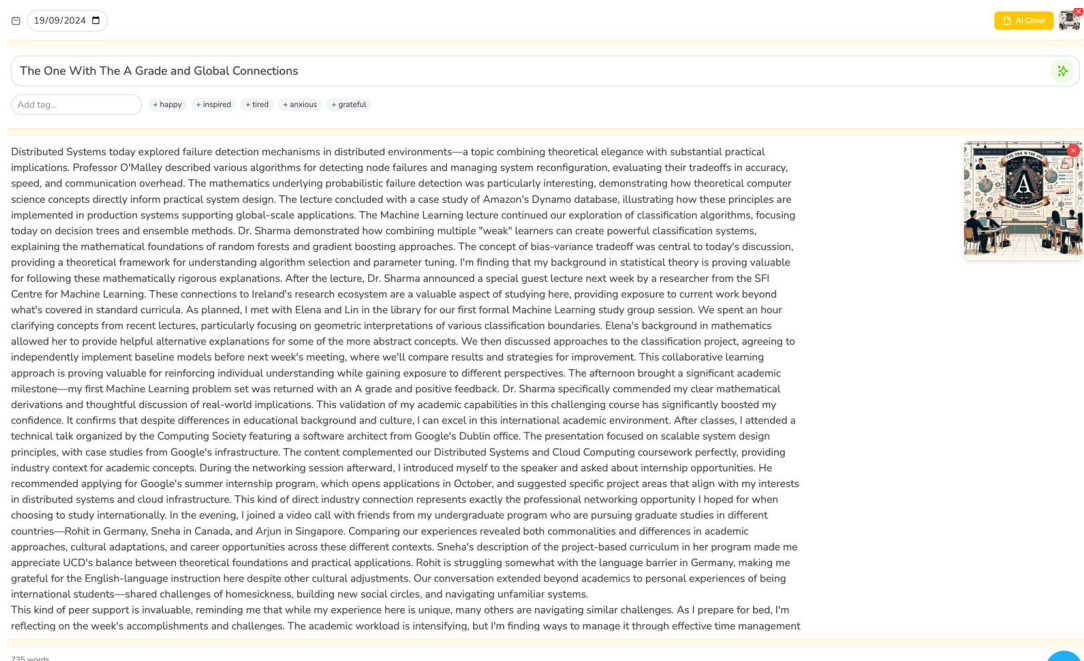


Figure 5: Example: A journal entry showing the AI-generated title and image.

- **Value Add vs. ChatGPT:** This automates creative tasks directly tied to the journal content, enriching the entry without requiring manual prompting from the user in a separate tool. The two-stage image generation likely produces more relevant images than a direct prompt from the short entry text.

These examples demonstrate how JournalMind's specific design choices (advanced retrieval pipeline, Function Calling, tailored prompts, automated workflows) create a journaling experience that is significantly more interactive, insightful, and engaging than using a generic LLM.

6 Summary and Conclusions

JournalMind successfully demonstrates the potential of integrating advanced LLM capabilities, specifically the OpenAI Assistants API, into a domain-specific application to provide significant value beyond generic chatbot interactions. By combining the `gpt-4o-mini` model with a custom advanced retrieval pipeline using a vector database, and leveraging the API's Function Calling feature, JournalMind offers users a powerful tool for self-reflection and personal insight based on their own journal entries.

The project achieved its core goals: creating a secure, user-friendly journaling platform enhanced with AI-driven analysis (sentiment, emotion), personalized Q&A grounded in user history, structured data retrieval via functions, and engaging creative features like F.R.I.E.N.D.S.-style titles and content-specific image generation. The design choices, particularly the custom advanced retrieval implementation and the use of Function Calling for specific tasks, proved effective in overcoming the limitations of standard LLMs for this personalized domain.

Future Work and Potential Enhancements: While JournalMind provides a robust foundation, several avenues exist for future development:

- **Advanced Analytics:** Implement tracking of mood trends over time, identify recurring themes or topics, and provide more sophisticated long-term summaries.
- **Expanded Functionality:** Add more function calls for different types of analysis (e.g., "Show me entries where I felt most productive," "Summarize my travel experiences").
- **Proactive Insights:** Develop features where the assistant proactively offers insights or prompts for reflection based on recent entries or detected patterns.
- **Improved Image Generation:** Refine the two-stage image generation prompts for even more accurate and aesthetically pleasing results. Explore different image models.
- **Data Export/Import:** Allow users to export their journal data or import existing journals.
- **Deployment and Scalability:** Refine the architecture and deployment strategy (e.g., using cloud services for databases, vector store, and hosting) for scalability and robustness.
- **Refined User Interface:** Continuously improve the UI/UX based on user feedback.

In conclusion, JournalMind serves as a compelling example of a specialized LLM assistant. It effectively utilizes the Assistants API while tailoring its capabilities through custom data handling, an advanced retrieval pipeline, and tool integration, resulting in a unique and valuable application for personal journaling and self-discovery. The project highlights the power of combining LLMs with thoughtful application design and domain-specific data management.

7 Acknowledgements

This project was undertaken as part of the COMP47980 module. We would like to thank Professor Tony Veale for his guidance and lectures, which enhanced our understanding of Generative AI and Language models. We would also like to thank our TAs, without whose help this project would not have been possible.

We acknowledge that the work presented in this report is entirely our own, and that every sentence in this report was written by us and us alone. Where ideas or text from others have been used, they are clearly marked with quotation marks and cited appropriately inline and in the References section. Plagiarism is a serious academic infraction, and care has been taken to avoid any ambiguity regarding the sources of information and inspiration.

8 References

- [1] OpenAI. (2024). *OpenAI API Documentation*. Retrieved from <https://platform.openai.com/docs>
- [2] FastAPI. (n.d.). *FastAPI Documentation*. Retrieved from <https://fastapi.tiangolo.com/>
- [3] React. (n.d.). *React Documentation*. Retrieved from <https://reactjs.org/docs/getting-started.html>
- [4] Tortoise ORM. (n.d.). *Tortoise ORM Documentation*. Retrieved from <https://tortoise-orm.readthedocs.io/>
- [5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474. (If relevant)